PL/pgSQL
Динамические команды



Авторские права
© Postgres Professional, 2017–2024
Авторы: Егор Рогов, Павел Лузанов, Илья Баштанов, Игорь Гнатюк
Фото: Олег Бартунов (монастырь Пху и пик Бхрикути, Непал)

Использование материалов курса
Некоммерческое использование материалов курса (презентации, демонстрации) разрешается без ограничений. Коммерческое использование возможно только с письменного разрешения компании Postgres Professional. Запрещается внесение изменений в материалы курса.

Обратная связь
Отзывы, замечания и предложения направляйте по адресу:
edu@postgrespro.ru

Отказ от ответственности
Компания Postgres Professional не несет никакой ответственности за любые повреждения и убытки, включая потерю дохода, нанесенные прямым или непрямым, специальным или случайным использованием материалов курса. Компания Postgres Professional не предоставляет каких-либо гарантий на материалы курса. Материалы курса предоставляются на основе принципа «как есть» и компания Postgres Professional не обязана предоставлять сопровождение, поддержку, обновления, расширения и изменения.

Темы

Причины использования

Выполнение динамического запроса

Способы формирования динамического запроса

Динамический SQL

Текст SQL-команды формируется в момент выполнения

Причины использования

- дополнительная гибкость в приложении
- формирование нескольких конкретных запросов вместо одного универсального для оптимизации

Цена

- операторы не подготавливаются
- возрастает риск внедрения SQL-кода
- возрастает сложность сопровождения

Под динамическими командами понимаются команды SQL, текст которых формируется и затем выполняется внутри PL/pgSQL-блока в подпрограммах или в анонимных блоках.

В большинстве случаев можно обойтись без динамических команд, но иногда они могут предоставить дополнительную гибкость. Например, можно встроить в определенные места приложения возможность выполнять команды, считанные из настроек системы. Управлять такими настройками могут специалисты поддержки во время эксплуатации приложения, а не программисты в момент разработки.

Иногда, при формировании отчета с большим количеством необязательных параметров, бывает проще формировать текст запроса прямо во время выполнения только для указанных параметров, чем заранее, при разработке, писать сложный запрос, учитывающий все возможные комбинации параметров.

Платой за использование динамических команд будет отказ от подготовленных операторов, которые по умолчанию используются в PL/pgSQL. Также нужно следить за безопасностью динамических команд с точки зрения возможности внедрения SQL-кода.

Следует отметить и существенное возрастание сложности сопровождения. В частности, в исходном коде приложения невозможно будет текстовым поиском обнаружить все выполняемые команды.

Выполнение запроса

Оператор EXECUTE

- выполняет строковое представление SQL-запроса
- позволяет использовать параметры
- переменные PL/pgSQL не становятся неявными параметрами

Может использоваться вместо SQL-запроса

- сам по себе
- при открытии курсора
- в цикле по запросу
- в предложении RETURN QUERY

Для выполнения динамических команд в PL/pgSQL используется команда EXECUTE, которая выполняет SQL-оператор, заданный в виде текстовой строки.

Динамический запрос может содержать явно заданные параметры. В тексте команды параметры обозначаются как $1, $2 и т. д., а значения параметров указываются в предложении USING. Параметры работают точно так же, как в подготовленных операторах (что рассматривалось в теме «Архитектура. Общее устройство PostgreSQL»). Однако переменные PL/pgSQL не становятся неявными параметрами, как это происходит при обычном, не динамическом, использовании SQL в PL/pgSQL.

Оператор EXECUTE может использоваться как самостоятельный оператор (в этом случае просто выполняется динамическая команда). Он также может использоваться в циклах по запросу, при открытии курсора, в команде RETURN QUERY — во всех этих случаях EXECUTE заменяет собой оператор SQL.

https://postgrespro.ru/docs/postgresql/16/plpgsql-statements#PLPGSQL-STATEMENTS-EXECUTING-DYN

Тонкий момент: процедура не может управлять транзакциями, если она вызывается оператором EXECUTE.

Выполнение динамического запроса

Оператор EXECUTE позволяет выполнить SQL-команду, заданную в виде строки.

```
=> DO $$
DECLARE
    cmd CONSTANT text := 'CREATE TABLE city_msk(
        name text, architect text, founded integer
    )';
BEGIN
    EXECUTE cmd; -- таблица для исторических зданий Москвы
END
$$;
```

```
DO
```

Предложение INTO оператора EXECUTE позволяет вернуть одну (первую) строку результата в переменную составного типа или несколько скалярных переменных.

Для проверки результата выполнения динамической команды можно использовать команду GET DIAGNOSTICS, как и в случае статических команд (но не переменную FOUND).

```
=> DO $$
DECLARE
    rec record;
    cnt bigint;
BEGIN
    EXECUTE 'INSERT INTO city_msk (name, architect, founded) VALUES
                ((''Пашков дом'', ''Василий Баженов'', 1784),
                (''Ансамбль Царицыно'', ''Василий Баженов'', 1776),
                (''Усадьба Тутолмина'', ''Василий Баженов'', 1788),
                (''Музей Пушкина'', ''Роман Клейн'', 1898),
                (''ЦУМ'', ''Роман Клейн'', 1908)
            RETURNING name, architect, founded'
    INTO rec;
    RAISE NOTICE '%', rec;
    GET DIAGNOSTICS cnt := ROW_COUNT;
    RAISE NOTICE 'Добавлено строк: %', cnt;
END
$$;
```

```
NOTICE:  ("Пашков дом","Василий Баженов",1784)
NOTICE:  Добавлено строк: 5
DO
```

При необходимости с помощью STRICT можно гарантировать, что команда обработает ровно одну строку.

Результат динамического запроса можно обработать в цикле FOR.

```
=> DO $$
DECLARE
    rec record;
BEGIN
    FOR rec IN EXECUTE 'SELECT * FROM city_msk WHERE architect = ''Роман Клейн'' ORDER BY founded'
    LOOP
        RAISE NOTICE '%', rec;
    END LOOP;
END
$$;
```

```
NOTICE:  ("Музей Пушкина","Роман Клейн",1898)
NOTICE:  (ЦУМ,"Роман Клейн",1908)
DO
```

Этот же пример с использованием курсора.

```
=> DO $$
DECLARE
    cur refcursor;
    rec record;
BEGIN
    OPEN cur FOR EXECUTE 'SELECT * FROM city_msk WHERE architect = ''Роман Клейн'' ORDER BY founded';
    LOOP
        FETCH cur INTO rec;
        EXIT WHEN NOT FOUND;
        RAISE NOTICE '%', rec;
    END LOOP;
END
$$;

NOTICE:  ("Музей Пушкина","Роман Клейн",1898)
NOTICE:  (ЦУМ,"Роман Клейн",1908)
DO
```

Оператор RETURN QUERY для возврата строк из функции также может использовать динамические запросы. Напишем функцию, возвращающую все здания, возведенные архитектором до определенной даты. Для этого нам понадобятся параметры:

```
=> CREATE FUNCTION sel_msk(architect text, founded integer DEFAULT NULL)
RETURNS SETOF text
AS $$
DECLARE
    -- параметры пронумерованы: $1, $2...
    cmd text := '
        SELECT name FROM city_msk
        WHERE architect = $1 AND ($2 IS NULL OR founded < $2)';
BEGIN
    RETURN QUERY
        EXECUTE cmd
        USING architect, founded; -- указываем значения по порядку
END
$$ LANGUAGE plpgsql;

CREATE FUNCTION

=> SELECT * FROM sel_msk('Роман Клейн');

    sel_msk
---------------
 Музей Пушкина
 ЦУМ
(2 rows)


=> SELECT * FROM sel_msk('Роман Клейн', 1905 );  -- до событий на Красной Пресне

    sel_msk
---------------
 Музей Пушкина
(1 row)
```

Формирование команды

Подстановка значений параметров

- предложение USING
- гарантируется невозможность внедрения SQL-кода

Экранирование значений

- идентификаторы: `format('%I')`, `quote_ident`
- литералы: `format('%L')`, `quote_literal`, `quote_nullable`
- внедрение SQL-кода невозможно при правильном использовании

Обычные строковые функции

- конкатенация и др.
- возможно внедрение SQL-кода!

Использование оператора EXECUTE имеет смысл, если команда формируется динамически. Приведенные ранее примеры можно было бы записать и без оператора EXECUTE.

Поскольку команда представляется текстовой строкой, ее можно сформировать обычными строковыми функциями, такими как конкатенация и т. п. В этом случае надо проявлять крайнюю осторожность, поскольку возможно внедрение SQL-кода.

Внедрение SQL-кода невозможно в принципе, если значения передаются как параметры с помощью предложения USING.

Однако параметры применимы не всегда: может понадобиться конкатенировать отдельные части запроса или подставлять в запрос имя таблицы. В этом случае для защиты от внедрений следует экранировать значения, полученные из ненадежного источника.

Для идентификаторов используется функция format со спецификатором %I или функция quote_ident. Эти функции формируют правильные имена идентификаторов, при необходимости заключая их в двойные кавычки и экранируя специальные символы.

Для подстановки литералов внутрь текста команды можно использовать функции quote_literal, quote_nullable или функцию format со спецификатором %L.

https://postgrespro.ru/docs/postgresql/16/functions-string

Возможность внедрения SQL-кода

Перепишем функцию, возвращающую здания, добавив параметр — код города. По задумке такая функция должна позволять обращаться только к таблицам, начинающимся на city_.

```
=> CREATE FUNCTION sel_city(
    city_code text,
    architect text,
    founded integer DEFAULT NULL
)
RETURNS SETOF text AS $$
DECLARE
    cmd text := '
        SELECT name FROM city_' || city_code || '
        WHERE architect = $1 AND ($2 IS NULL OR founded < $2)';
BEGIN
    RAISE NOTICE '%', cmd;
    RETURN QUERY
        EXECUTE cmd
        USING architect, founded;
END
$$ LANGUAGE plpgsql;
```

```
CREATE FUNCTION
```

Функция правильно работает при «нормальных» значениях параметров:

```
=> SELECT * FROM sel_city('msk', 'Василий Баженов');
```

```
NOTICE:
        SELECT name FROM city_msk
        WHERE architect = $1 AND ($2 IS NULL OR founded < $2)
     sel_city
------------------
 Пашков дом
 Ансамбль Царицыно
 Усадьба Тутолмина
(3 rows)
```

Однако злоумышленник может подобрать такое значение, которое изменит синтаксическую конструкцию запроса и позволит ему получить несанкционированный доступ к данным:

```
=> SELECT * FROM sel_city('msk WHERE false
        UNION ALL
        SELECT usename FROM pg_user
        UNION ALL
        SELECT name FROM city_msk', '');
```

```
NOTICE:
        SELECT name FROM city_msk WHERE false
        UNION ALL
        SELECT usename FROM pg_user
        UNION ALL
        SELECT name FROM city_msk
        WHERE architect = $1 AND ($2 IS NULL OR founded < $2)
 sel_city
----------
 student
 postgres
(2 rows)
```

При использовании подготовленных операторов или динамических команд с параметрами это невозможно в принципе, так как структура SQL-запроса фиксируется при синтаксическом разборе. Выражение всегда останется выражением и не сможет превратиться, например, в имя таблицы.

Формирование динамической команды

Параметры в предложении USING нельзя использовать для имен объектов (названия таблиц, столбцов и пр.) в динамической команде. Такие идентификаторы необходимо экранировать, чтобы структура запроса не могла измениться:

```
=> SELECT format('%I', 'foo'),
          format('%I', 'foo bar'),

          format('%I', 'foo"bar');

 format |  format   |   format
--------+-----------+------------
 foo    | "foo bar" | "foo""bar"
(1 row)
```

То же самое выполняет и другая функция:

```
=> SELECT quote_ident('foo'),
          quote_ident('foo bar'),
          quote_ident('foo"bar');

 quote_ident | quote_ident | quote_ident
-------------+-------------+-------------
 foo         | "foo bar"   | "foo""bar"
(1 row)
```

Вот как может выглядеть пример с созданием таблицы:

```
=> DO $$
DECLARE
    cmd CONSTANT text := 'CREATE TABLE %I(
        name text, architect text, founded integer
    )';
BEGIN
    EXECUTE format(cmd, 'city_spb'); -- таблица для Санкт-Петербурга
    EXECUTE format(cmd, 'city_nov'); -- таблица для Новгорода
END
$$;

DO
```

Вместо использования параметров, можно вставлять в строку литералы. В этом случае также требуется экранирование, но другое:

```
=> SELECT format('%L', 'foo bar'),
          format('%L', 'foo''bar'),
          format('%L', NULL);

  format   |   format   | format
-----------+------------+--------
 'foo bar' | 'foo''bar' | NULL
(1 row)
```

Это же выполняет и функция quote_nullable:

```
=> SELECT quote_nullable('foo bar'),
          quote_nullable('foo''bar'),
          quote_nullable(NULL);

 quote_nullable | quote_nullable | quote_nullable
----------------+----------------+----------------
 'foo bar'      | 'foo''bar'     | NULL
(1 row)
```

Похожая функция quote_literal отличается тем, что не превращает неопределенное значение в литерал:

```
=> SELECT quote_literal(NULL);

 quote_literal
---------------

(1 row)
```

В качестве примера перепишем функцию, возвращающую здания в определенном городе, без использования параметров, но так, чтобы она осталась безопасной.

```
=> CREATE OR REPLACE FUNCTION sel_city(
    city_code text,
    architect text,
    founded integer DEFAULT NULL
)
RETURNS SETOF text

AS $$
DECLARE
    cmd text := '
        SELECT name FROM %I
        WHERE architect = %L AND (%L IS NULL OR founded < %L::integer)';
BEGIN
    RETURN QUERY EXECUTE format(
        cmd, 'city_'||city_code, architect, founded, founded
    );
END
$$ LANGUAGE plpgsql;

CREATE FUNCTION
```

Обратите внимание, что в этом случае получается два лишних приведения типов: сначала параметр типа integer приводится к строке, а затем, на этапе выполнения, строка обратно приводится к integer (в случае использования параметров USING такого не происходит):

```
=> SELECT * FROM sel_city('msk', 'Василий Баженов', 1785);  -- до приезда императрицы в Царицыно

     sel_city
------------------
 Пашков дом
 Ансамбль Царицыно
(2 rows)
```

Попытка передачи некорректного значения не приведет к успеху:

```
=> SELECT * FROM sel_city('msk WHERE false
        UNION ALL
        SELECT usename FROM pg_user
        UNION ALL
        SELECT name FROM city_msk', '');

NOTICE:  identifier "city_msk WHERE false
        UNION ALL
        SELECT usename FROM pg_user
        UNION ALL
        SELECT name FROM city_msk" will be truncated to "city_msk WHERE false
        UNION ALL
        SELECT usename F"
ERROR:  relation "city_msk WHERE false
        UNION ALL
        SELECT usename F" does not exist
LINE 2:         SELECT name FROM "city_msk WHERE false
                                ^
QUERY:
        SELECT name FROM "city_msk WHERE false
        UNION ALL
        SELECT usename FROM pg_user
        UNION ALL
        SELECT name FROM city_msk"
        WHERE architect = '' AND (NULL IS NULL OR founded < NULL::integer)
CONTEXT:  PL/pgSQL function sel_city(text,text,integer) line 7 at RETURN QUERY
```

Итоги

Динамические команды дают дополнительную гибкость

Позволяют формировать отдельные запросы для разных значений параметров с целью оптимизации

Не подходят для коротких, частых запросов

Увеличивается сложность поддержки